CS 409 : Introduction to Cryptography Fall 2025

Assignment 3
CCA, MAC and Hash

Instructor: Sruthi Sekar *TAs:* Lakshya Gadhwal, Ravi B Prakash

Problem 1: Lower Bound on Fixed-Length-Tag MAC

Let $\Pi = (\mathsf{Gen}, \mathsf{Mac}, \mathsf{Vrfy})$ be an EU-CMA secure MAC (Lec 8, Def 2), and for $k \in \{0,1\}^n$ the tag-generation algorithm Mac_k always outputs tags of length $t(n)$. Prove that t must be super-logarithmic or, equivalently, that if $t(n) = \mathcal{O}(\log n)$ then Π cannot be an EU-CMA secure MAC. **Hint:** Consider the probability of randomly guessing a valid tag.

Problem 2: Am I Secure?

Let $F = \{f_k : \{0,1\}^n \to \{0,1\}^n\}_{k\in\{0,1\}^n}$ be a pseudorandom function (c.f. Lec 6, Def 1 or 2). Consider the MAC for messages of length $\ell(n) = 2n - 2$:

- $\mathsf{Gen}(1^n)$: outputs $k \in_R \{0,1\}^n$.
- $\mathsf{Mac}(k, m)$: on input a message $m = m_0||m_1$ (with $|m_0| = |m_1| = n - 1$) and key k, output $t = f_k(0||m_0)||f_k(1||m_1)$.

Vrfy is defined in the natural way. Is $(\mathsf{Gen}, \mathsf{Mac}, \mathsf{Vrfy})$ EU-CMA secure? Prove your answer.

Problem 3: On Insecurity

Let $F = \{f_k : \{0,1\}^n \to \{0,1\}^n\}_{k\in\{0,1\}^n}$ be a pseudorandom function. Show that each of the following MACs is insecure, even if used to authenticate fixed-length messages. (In each case $\mathsf{Gen}(1^n)$ outputs a uniform $k \in \{0,1\}^n$). Let $\langle i \rangle$ denote an $n/2$-bit encoding of the integer i.)

a) $\mathsf{Mac}_1(k, m)$: To authenticate a message $m = m_1, \ldots, m_\ell$, where $m_i \in \{0,1\}^n$, output $t := f_k(m_1) \oplus \cdots \oplus f_l(m_\ell)$.

b) $\mathsf{Mac}_2(k, m)$: To authenticate a message $m = m_1, \ldots, m_\ell$, where $m_i \in \{0,1\}^{n/2}$, output $t := f_k(\langle 1 \rangle, m_1) \oplus \cdots \oplus f_l(\langle \ell \rangle, m_\ell)$.

c) $\mathsf{Mac}_3(k, m)$: To authenticate a message $m = m_1, \ldots, m_\ell$, where $m_i \in \{0,1\}^{n/2}$, pick $r \in_R \{0,1\}^n$, compute $t := f_k(r) \oplus f_k(\langle 1 \rangle, m_1) \oplus \cdots \oplus f_k(\langle \ell \rangle, m_\ell)$, and output (r, t).

In each case, Vrfy is the natural one.

Problem 4: Could I *Be* More Insecure?

Let $F = \{f_k : \{0,1\}^n \to \{0,1\}^n\}_{k\in\{0,1\}^n}$ be a pseudorandom function. Show that the following MAC for messages of length $2n$ is insecure:

- $\mathsf{Gen}(1^n)$: outputs a uniform $k \in_R \{0,1\}^n$.
- $\mathsf{Mac}(k, m)$: To authenticate a message $m = m_1||m_2$ with $|m_1| = |m_2| = n$, output the tag $t := f_k(m_1)||f_k(f_k(m_2))$.

Vrfy just re-runs the Mac to check (canonical verification).

Problem 5: The CBC-MAC (Cousins) Forge

We explore what happens when the basic CBC-MAC construction (c.f. Lec 9) is modified in different ways.

a) Say the sender and receiver do not agree on the message length in advance and in CBC-MAC, the length is not pre-pended. Then, $\mathsf{Vrfy}_k(m,t) = 1$ iff $t \stackrel{?}{=} \mathsf{Mac}_k(m)$, regardless of the length of m. However, the sender is careful to only authenticate messages of length $2n$. Show that an adversary can forge a valid tag on a message of length $4n$.

b) Let F be a keyed function that is an EU-CMA secure (deterministic) MAC for messages of length n (i.e., $F_k = \mathsf{Mac}_k$, for $k \in \{0,1\}^n$). Note that F need not be a pseudorandom function. Show that CBC-MAC instantiated with this F is not necessarily EU-CMA secure. (**Hint**: try constructing a secure MAC, modifying the PRF-based construction, that helps with the attack)

Problem 6: Authenticate-then-Encrypt

Prove that the authenticate-then-encrypt approach (Lec 8, Attempt II), instantiated with any CPA-secure encryption scheme and any EU-CMA secure MAC, yields a CPA-secure encryption scheme that has ciphertext integrity.

Problem 7: OR Constructions

Let (Gen_1, H_1) and (Gen_2, H_2) be two hash functions. Define (Gen, H) so that Gen runs Gen_1 and Gen_2 to obtain keys k_1 and k_2, respectively. Then define $H^{k_1,k_2}(x) = H_1^{k_1}(x)||H_2^{k_2}(x)$. Prove that if at least one of (Gen_1, H_1) and (Gen_2, H_2) is collision resistant, then (Gen, H) is collision resistant. (c.f. Lec 10, Def 1).

Problem 8: Self-Composition of CRHF

Let (Gen, H) be a collision-resistant hash function. Is $(\mathsf{Gen}, \hat{H})$ defined by $\hat{H}^k(x) := H^k(H^k(x))$ necessarily collision resistant?

Problem 9: Mac from Hash

Let (Gen, H) be collision resistant. Consider the following MAC scheme for arbitrary-length messages:

- $\mathsf{Gen}_M(1^n)$: $k \leftarrow \mathsf{Gen}(1^n)$, $r \in_R \{0,1\}^n$. output (k, r).
- $\mathsf{Mac}_{k,r}(m)$: output $H^k(r||m)$.

Show that this is not an EU-CMA secure MAC when (Gen, H) is constructed via the Merkle-Damgård transform (c.f. Lec 10, Construction 2). Note here that the hash-key k is not hidden from the MAC adversary, only r is hidden!

Problem 10: Let's Modify Merkle Damgård!

Recall the Merkle-Damgård transform (c.f. Lec 10, Construction 2):

Merkle-Damgård

Let (Gen, h) be a fixed-length hash function for inputs of length $2n$ and with output length n. Construct the hash function (Gen, H), where Gen remains unchanged and H is as follows:
H: on input a key k and a string $x \in \{0,1\}^*$ of length $|x| < 2^n$, do the following:

1. Set $\ell := \left\lceil \frac{|x|}{n} \right\rceil$ (i.e., the number of blocks in x). Pad x with zeroes so its length is a multiple of n. Parse the padded result as the sequence of n-bit blocks $x_1, \ldots, x_\ell$. Set $x_{\ell+1} := |x|$, where $|x|$ is encoded as an n-bit string.
2. Set $z_0 := 0^n$. (This is also called the IV.)
3. For $i = 1, \ldots, \ell + 1$, compute $z_i := h^k(z_{i-1}||x_i)$ and output $z_{\ell+1}$.

For each of the following modifications to the Merkle-Damgård transform, determine whether the result is collision resistant. If yes, provide a proof; if not, demonstrate an attack.

a) Modify the construction so that the input length is not included at all (i.e., output z_ℓ and not $z_{\ell+1} = h^k(z_\ell|||x|)$). (Assume the resulting hash function is only defined for inputs whose length is an integer multiple of the block length.)

b) Modify the construction so that instead of outputting $z = h^k(z_\ell|||x|)$, the algorithm outputs $z_\ell|||x|$.

c) Instead of using an IV, just start the computation from x_1. That is, define $z_1 := x_1$ and then compute $z_i := h^k(z_{i-1}||x_i)$ for $i = 2, \ldots, \ell + 1$ and output $z_{\ell+1}$ as before.

d) Instead of using a fixed IV, set $z_0 := |x|$ and then compute $z_i := h^k(z_{i-1}||x_i)$ for $i = 1, \ldots, \ell$ and output z_ℓ.